

Monitoramento e otimização de recursos em aplicativos Flutter

Beatriz Vocurca Frade

Universidade Federal de Minas Gerais
Engenharia de Sistemas
Orientador: Prof. Ramon Lacerda Marques
2026



Roteiro da defesa

20 minutos

1. Problema

Contexto Flutter, motivação e pergunta de pesquisa.

2. Base

Literatura, ferramentas existentes e lacuna.

3. Solução

`collector_flutter`, arquitetura, métricas e dashboard.

4. Método

Cenários controlados, evidências e limites.

5. Resultados

Sinais observados, ADB, 60 testes e contribuições.

6. Fechamento

Limitações, trabalhos futuros e conclusão.

Ponto de partida

Introdução

- Flutter acelera o desenvolvimento multiplataforma.
- Apps modernos combinam animações, listas dinâmicas, rede, estado reativo e dispositivos muito diferentes.
- Desempenho ruim aparece como *jank*, queda de FPS, uso alto de memória, rede excessiva e bateria menor.
- O diagnóstico precisa acontecer enquanto o app ainda está sendo construído.

FPS

Jank

Memória

Rede

Problema de pesquisa

Ferramentas fortes, uso com atrito

O que já existe

Flutter DevTools, *Android Profiler*, *Perfetto*, *Instruments* e *Firebase* entregam dados ricos para profiling e diagnóstico.

O que dificulta o uso contínuo

Ferramentas externas exigem configuração, troca de contexto e interpretação manual dos dados.

Como tornar o diagnóstico de desempenho mais acessível, contínuo e acionável em Flutter?

Objetivos

O que o trabalho se propôs a entregar

Objetivo geral

Desenvolver e avaliar funcionalmente o `collector_flutter`, uma biblioteca Dart/Flutter embarcada para coletar métricas, analisar sintomas de degradação e exibir recomendações em tempo real.

- Coletar FPS, *jank*, memória, rede, CPU, bateria, eventos e *rebuilds*.
- Organizar o pacote em arquitetura modular.
- Exibir um dashboard dentro do app.
- Gerar recomendações heurísticas por severidade.
- Exportar sessões em JSON e CSV.
- Validar o comportamento em cenários controlados.

Base científica

O que a literatura ajudou a decidir

Energia

Estudos como Pathak, Hao, Li e Hoque relacionam consumo a CPU, rede, execução e memória.

Performance móvel

AppInsight e Rua/Saraiva reforçam análise de múltiplos sinais, não apenas uma métrica isolada.

Diagnóstico

Sun e Bangash apoiam a ideia de transformar sintomas observáveis em hipóteses de otimização.

Síntese: desempenho móvel é multidimensional; a ferramenta precisa observar vários sinais ao mesmo tempo.

Lacuna identificada

Entre medir e agir

Ferramentas de referência

- Medições detalhadas.
- Ótimas para investigação profunda.
- Importantes como referência técnica.

Espaço do trabalho

- Feedback dentro do app.
- Baixa fricção durante o desenvolvimento.
- Recomendações iniciais para orientar ação.

O ponto aqui não é substituir o Profiler; é aproximar o primeiro diagnóstico do fluxo diário de desenvolvimento.

Humanidades

Por que performance também é impacto

Experiência

Apps lentos reduzem confiança, fluidez e sensação de qualidade.

Inclusão

Eficiência ajuda o app a funcionar melhor em aparelhos de entrada ou mais antigos.

Ética

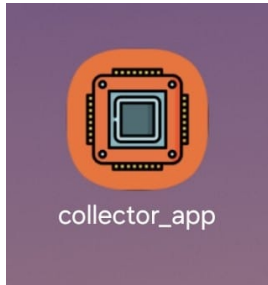
O pacote coleta métricas técnicas, sem dados pessoais ou conteúdo do usuário.

Otimizar recursos é também respeitar bateria, dispositivo e contexto de uso.

Proposta: collector_flutter

Termômetro embarcado

- Pacote Dart/Flutter de código aberto.
- Coleta métricas dentro da própria aplicação monitorada.
- Analisa sintomas por heurísticas e severidade.
- Exibe recomendações em um dashboard integrado.
- Exporta dados para inspeção externa.



Em uma frase

A ferramenta funciona como um termômetro embarcado: observa sinais vitais do app e mostra onde investigar primeiro.

Arquitetura

Três camadas

Data

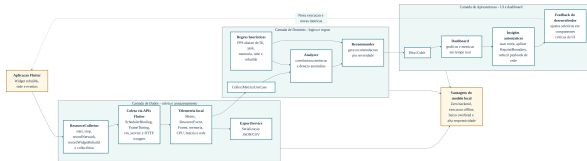
Fontes de dados para frames, memória, rede, eventos, CPU e bateria.

Domain

Entidades, casos de uso, Analyzer e Recommender.

Presentation

CollectorBloc, DashboardPage e visualização reativa.

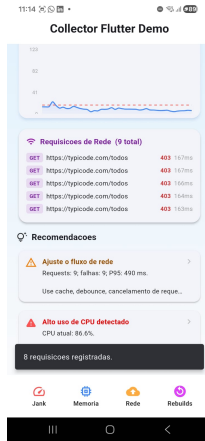


Dashboard integrado

Evidência visual do pacote



Estado estável

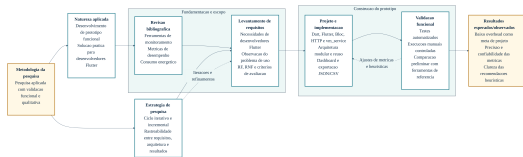


Gargalos detectados

Metodologia

Avaliação funcional

- Pesquisa aplicada: constrói uma solução prática.
- Ciclo iterativo: revisão, requisitos, implementação e validação.
- Execuções controladas no aplicativo de exemplo.
- Resultados tratados como evidência funcional, não estatística.



Cenários testados

Quatro cargas controladas

1. Rebuilds	2. Rede	3. Memória	4. Combinado
Reconstruções intensivas de interface.	Requisições HTTP simultâneas.	Alocação intensiva de objetos.	Interface, rede, memória e CPU.

Cada cenário provoca um sintoma esperado; o painel precisa tornar esse sintoma visível.

Evidências dos cenários

Imagens geradas no projeto



Rebuilds



Rede



Memória



Carga combinada

As capturas foram preservadas na monografia como Apêndice B, junto das leituras ADB.

Resultados observados

O que apareceu nos logs e no painel

Cenário	Sinais registrados	Leitura para a defesa
Rebuilds	30 <i>rebuilds</i> , FPS 55,4, P95 20,4 ms e 2 frames longos.	O painel reagiu à carga de interface.
Rede	9 requisições, FPS 42,9, P95 139,5 ms e 4 frames longos.	O interceptador registrou eventos e latências.
Memória	ADB: RSS 236.480 KB e PSS 222.232 KB.	Houve crescimento real, mas o snapshot do painel mostrou limitação de leitura.
Combinado	17 requisições, 54 <i>rebuilds</i> , FPS 44,6, P95 37,7 ms e CPU 75,3%.	O sistema processou múltiplos sinais e priorizou alertas.

Essa evidência mostra operacionalidade, não precisão estatística.

ADB, Profiler e limites

Transparência metodológica

O que foi usado de fato

- `adb shell dumpsys meminfo`
- `adb shell dumpsys gfxinfo`
- Logs EVIDENCE do aplicativo
- Capturas reais do emulador

Como interpretar

Android Profiler e Perfetto ficaram como referências conceituais. Nesta avaliação, não houve série comparativa quantitativa nem cálculo formal de erro percentual.

Limitação importante

O `gfxinfo` confirmou a superfície Flutter, mas retornou contadores HWUI zerados nessa execução.

Testes automatizados

60 casos aprovados

60 testes

- 24 em análise e estatísticas.
- 17 em CPU e bateria.
- 12 em recomendações.
- 7 em exportação.

Arquivo	Cobertura	Casos
Analyzer		24
CPU/bateria		17
Recommender		12
ExportService		7

Papel dos testes

Eles verificam regras internas, serialização e fallbacks. Não substituem experimentos quantitativos em dispositivos reais.

Limitações e trabalhos futuros

O que fica para evoluir

Limitações

- Validação funcional, sem análise estatística.
- Foco principal em Android.
- Sem estudo formal de usabilidade.
- Comparação ADB apenas como apoio qualitativo.

Trabalhos futuros

- Rodadas quantitativas com múltiplos dispositivos.
- Séries comparáveis com Profiler, Perfetto e ADB.
- CPU e energia mais completas no iOS.
- Avaliação com desenvolvedores Flutter.

Conclusão

Mensagem final

- O `collector_flutter` aproxima o primeiro diagnóstico de desempenho do fluxo real de desenvolvimento Flutter.
- A arquitetura em camadas permitiu evoluir o pacote com CPU, bateria, exportação e testes.
- Os quatro cenários controlados indicaram operacionalidade e recomendações compatíveis com os sintomas induzidos.
- A validação foi honesta sobre seus limites: funcional e qualitativa, com trabalhos futuros bem definidos.

Fechamento

Performance não é apenas fazer o app rodar mais rápido; é fazer o app respeitar melhor o dispositivo, a bateria e a pessoa que o usa.

Perguntas?

collector_flutter

UF *m* G



Beatriz Vocurca Frade | Engenharia de Sistemas – UFMG